Table 5: The accuracy of each condition's preference dataset from the TRAINED experiment with respect to the *noiseless* version of the target preference model. Higher is better. See Table 4 for more details on how to interpret this table.

Condition	Noiseless-P_{Σ_r} Accuracy	Noiseless-P_{regret} Accuracy
Control Condition	53.7%	68.8%
Influenced Towards Target Model	75.5%	75.4%

Table 6: The accuracy of each condition's preference dataset from the QUESTION experiment with respect to the *noiseless* version of the target preference model. Higher is better. See Table 4 for more details on how to interpret this table.

Condition	Noiseless-P_{Σ_r} Accuracy	Noiseless-P_{regret} Accuracy
Control Condition	54.8%	69.9%
Influenced Towards Target Model	65.2%	68.7%

J Learning reward functions from preferences

J.1 Design pattern for learning a reward function from preferences

We follow the general procedure for learning a reward function from a dataset of preferences depicted in Figure 23. This procedure is executed for all preference datasets in each experiment, which all share the same ground-truth reward function r.

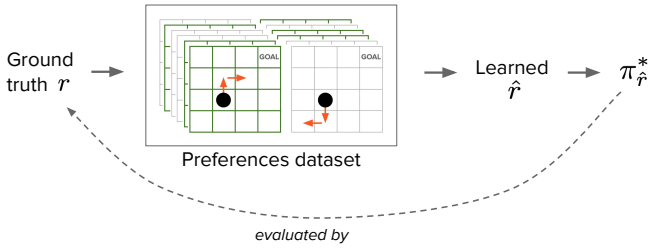

Figure 23: An outline of the general procedure for learning a reward function from a preference dataset and then evaluating that reward function. The generic gridworld shown is for illustrative purposes only. Figure provided by Knox et al. (2024).

J.2 Additional details for learning a reward function from preferences

Doubling the preference dataset by reversing preference samples When learning a reward function from a preference dataset, we double the amount of data by duplicating each preference sample and then flipping the preference label and segment pair ordering. This provides more training data and avoids learning any segment ordering effects.
The reward representation The ground-truth reward function r is assumed to be a linear combination of weights and features. Any reward function learned from a preference dataset $\hat{r}$ takes the same form. This linearity assumption enables us to use the tractable algorithm for learning a reward function with P_{regret} proposed by Knox et al. (2022).
Discounting during value iteration The delivery domain is an episodic environment but a policy derived from a poorly learned reward function can endlessly avoid terminal states, resulting in a return of negative infinity. Therefore during value iteration and when computing a policy's mean return with respect to r, we apply a discount factor of $\gamma = 0.999$. We chose this high discount factor to avoid returns of negative infinity while having a negligible effect on the returns of high-performing policies and still allowing value iteration to converge within a reasonable time.
Early stopping when learning with P_{regret} Knox et al. (2022) found that when learning a reward function using P_{regret}, the training loss tended to fluctuate cyclically. To handle this, they use the $\hat{r}$ that achieved the lowest loss during training instead of the final $\hat{r}$. We follow the same procedure.

J.3 Hyperparameters for learning a reward function from preferences

The following hyperparameters were used by Knox et al. (2024) and across all our experiments. See Knox et al. (2024) for more details on how they were chosen.

Reward learning with the partial return preference model
learning rate: 2; number of training epochs: $30,000$; and optimizer: Adam (with $\beta_1 = 0.9$ and $\beta_2 = 0.999$, and eps= $1e-08$).

Reward learning with the regret preference model
learning rate: 0.5; number of training epochs: $5,000$; optimizer: Adam (with $\beta_1 = 0.9$, $\beta_2 = 0.999$, and eps=$1e-08$); and softmax temperature: 0.001.

Additionally, learning with P_{regret} following the algorithm proposed by Knox et al. (2024) requires a set of successor features from candidate policies which are used to approximate $V^*_{\hat{r}}(.)$, a component of the regret preference model. Because we use the same delivery ask as Knox et al. (2024), we use the set of successor features that they generate.

When generating Figures 6, 8, 10, 25, and 26, we use random seeds $1-10$. The human preference datasets contain varying numbers of preferences (see Table 2). For all datasets within an experiment, we randomly subsample preferences to match the size of the smallest dataset—1793 preferences for the PRIVILEGED experiment datasets, 473 preferences for the TRAINED experiment datasets and 434 preferences for the QUESTION experiment datasets. This allows for easier comparison when partitioning the resulting datasets.

J.4 Computer specifications and software libraries used

The computer used to run all experiments had the following specification. Processor: 2.8 GHz Quad-Core Intel Core i7; Memory: 16 GB. Pytorch 2.0.1 (Paszke et al., 2019) was used to implement all reward learning models, and statistical analyses were performed using Scikit-learn 1.3.0 (Pedregosa et al., 2011).

J.5 Additional results for learning reward functions

Figure 24 complements Figure 6, Figure 25 complements Figure 8, and Figure 26 complements Figure 10, showing the percentage of partitions where the learned reward functions results in better performance than a policy that selects actions uniformly. In general, for each partition size, ranking each preference dataset by the percentage of better-than-random performance induced by the learned reward functions produces the same order as when using near-optimal performance.

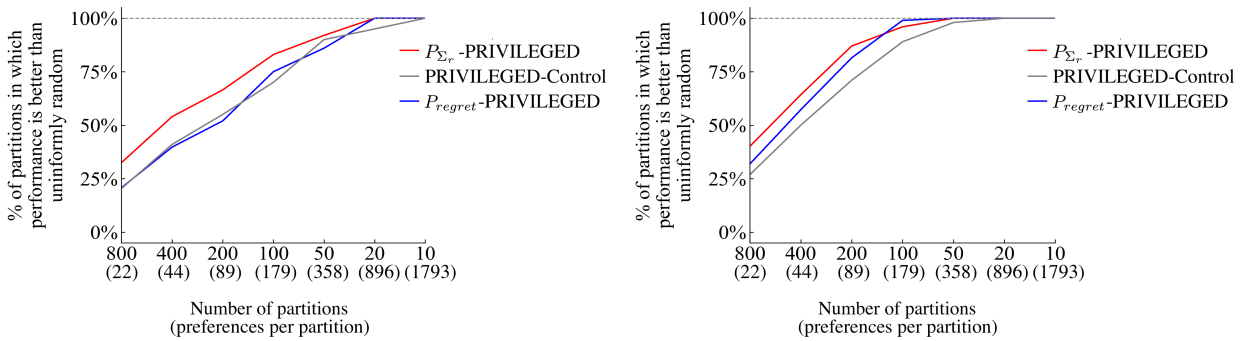

Figure 24: Learning a reward function with the partial return preference model (Left) and regret preference model (Right) from the preferences collected in the PRIVILEGED experiment. This figure complements Figure 6.

J.6 Addressing the partial return preference model's identifiablity issues

Learning with the partial return preference model from the P_{Σ_r}-TRAINED and TRAINED-Control datasets often fails to recover a reward function that induces near-optimal performance (see Figure 8). Knox et al. (2022) demonstrated that, in this grid-world domain, learning with the partial return preference model requires preference labels over pairs of trajectory segments in which one segment terminates earlier than the other at both the positive and negative terminal states (i.e., the inverted teardrop and sheep in Figure 3). Such segment pairs help mitigate the identifiability issue of the partial return preference model related to a constant shift in the reward function. To address this issue,

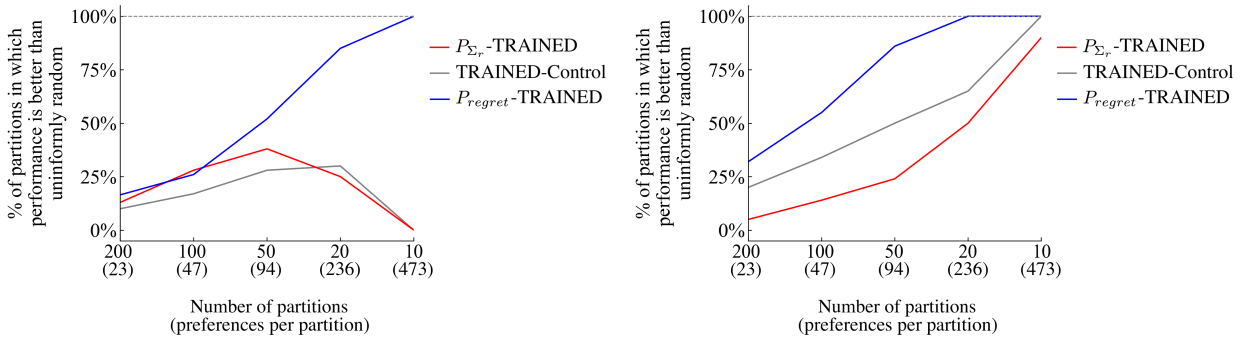

Figure 25: Learning a reward function with the partial return preference model (Left) and regret preference model (Right) from the preferences collected in the TRAINED experiment. This figure complements Figure 8.

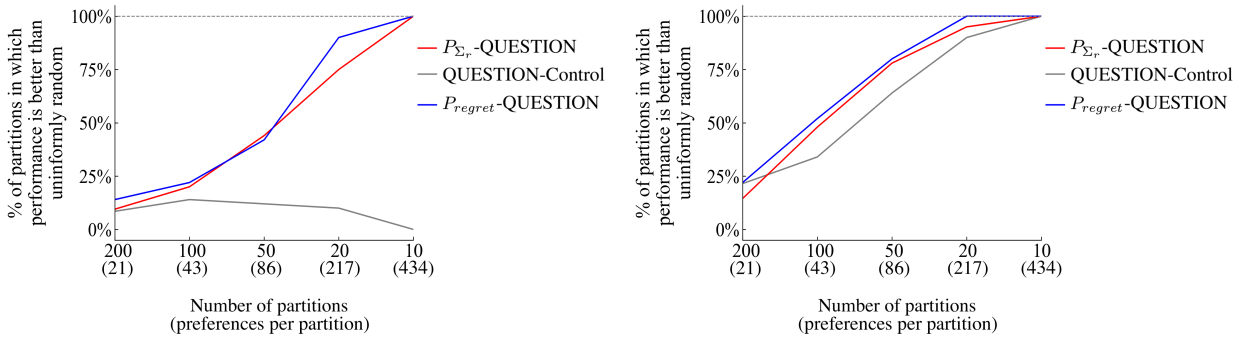

Figure 26: Learning a reward function with the partial return preference model (Left) and regret preference model (Right) from the preferences collected in the QUESTION experiment. This figure complements Figure 10.

Knox et al. (2022) manually selected types of segment pairs for preference labeling. We follow their segment pair selection methodology for the PRIVILEGED experiment, but for the TRAINED and QUESTION experiments, trajectory segments were constructed by randomly sampling actions to emulate a more realistic preference elicitation procedure. Consequently the partial return preference model may not recover the ground-truth reward function from the resulting preference datasets, which we hypothesize as explaining the partial return preference model's poor performance when learning from the P_{Σ_r}-TRAINED and TRAINED-Control datasets in Figure 8. Knox et al. (2024) showed that using preferences generated by P_{regret} to learn a reward function with the partial return preference model results in a reward function that is equivalent to an optimal advantage function, which may explain why the partial return preference model recovers performant reward functions from the P_{regret}-TRAINED dataset. We leave an investigation into this hypothesis to future work.

To empirically test whether the absence of specific segment pairs contributed to the poor performance of the partial return preference model when learning reward functions from the P_{Σ_r}-TRAINED and TRAINED-Control datasets, we added 50 additional segment pairs to each dataset. In these additional segment pairs, one segment terminates at the positive terminal state in fewer than three time-steps while the other segment does not terminate after three time-steps. These segment pairs would appear in the right-most graph in Figure 14, and were assigned preference labels by the partial return preference model with the ground-truth reward function. Figure 27 present the results when learning reward functions using these additional *synthetic* preferences.

Learning with the regret preference model using these additional preferences (bottom row of Figure 27) induces comparable results to those in Figure 8 and Figure 25 which matches our expectations; the regret preference model does not suffer from the same identifiability issues as the partial return preference model. Learning with the partial return preference model when including the additional preferences (top row of Figure 27) results in reward functions that induce near-optimal behavior more often for all datasets across all partition sizes. Including these preferences also results in better-than-uniformly-random behavior significantly more often across all partition sizes and datasets. These results therefore support our hypothesis that the partial return preference model's poor performance when learning from the P_{Σ_r}-TRAINED and TRAINED-Control datasets is—at least in part—due to the datasets missing specific segment pairs that account for the partial return preference model's identifiability issues.